codigo

Analise o arquivo em anexo passo a passo e determine ao final: este arquivo é benigno ou é um malware?

Aqui está a análise passo a passo do código-fonte em C fornecido (`codigo.c`):

Análise Passo a Passo

1. **Verificação de Privilégios e Inicialização**

   A função `main` inicia verificando se o usuário que está executando o programa possui privilégios de `root` (`getuid() != 0`). Caso contrário, o programa é encerrado imediatamente.

   O código cria e verifica um arquivo de controle de PID em `/var/run/haldrunner.pid` para garantir que apenas uma instância esteja rodando no sistema.

2. **Mascaramento de Processo (*Process Hiding / Spoofing*)**

   O programa altera o nome do processo e os argumentos (`argv[0]`) utilizando uma lista de daemons legítimos do sistema operacional (como `/sbin/udevd -d` , `dbus-daemon --system` , `/sbin/auditd -n` , etc.) de forma aleatória. Isso é feito por meio de manipulação direta de `environ` e da chamada `prctl(PR_SET_NAME)` . O objetivo é dificultar a detecção visual por administradores usando comandos como `ps` .

3. **Captura de Pacotes de Rede de Baixo Nível (*Packet Sniffing*)**

   Na função `packet_loop` , o programa abre um socket bruto (*raw socket*: `PF_PACKET, SOCK_RAW`) e aplica um filtro BPF (*Berkeley Packet Filter*) configurado via `setsockopt` .

   Ele monitora pacotes de rede (TCP, UDP e ICMP) em busca de um padrão específico chamado de **"pacote mágico"** (`magic_packet`), que funciona como um mecanismo de *port knocking* ou acionamento remoto oculto baseado em senha.

4. **Criptografia de Tráfego (RC4)**

   O código implementa algoritmos de criptografia de fluxo RC4 (`rc4_init` e `rc4`) junto com funções personalizadas de leitura e escrita (`cread` e `cwrite`). Todo o tráfego de dados e comandos trocados pela rede é criptografado para evitar a interceptação por ferramentas de monitoramento de rede comuns.

5. **Acesso Remoto e Backdoor (*Remote Shell*)**

   Quando o pacote mágico é validado com sucesso por meio de senhas embutidas no código (hashes que correspondem a strings como `"justforfun"` e `"socket"`), o programa entra em ação:

   - Manipula regras de firewall locais utilizando o comando `/sbin/iptables` para abrir portas ou redirecionar conexões (`getshell`).
   - Cria pseudo-terminais (`pty` / `tty`) e inicia uma shell interativa (`/bin/sh`), redirecionando fluxos de entrada, saída e erro para o socket remoto.
   - Permite ao atacante obter controle total do sistema operacional alvo remotamente.

Conclusão Final

O arquivo **é um malware** (especificamente, um **backdoor/rootkit para sistemas Linux**). Ele reúne características típicas de artefatos maliciosos avançados: persistência, ocultação de processos (*masquerading*), sniffing de pacotes em sockets brutos, criptografia de canal de comando e controle (C2) e abertura de acesso remoto não autorizado via shell interativa.

Qual é a categoria principal deste malware?

Com base na análise do código, a categoria principal deste malware é um **Backdoor** (com características avançadas de **Rootkit** para sistemas Linux).

Justificativa:

Acesso Remoto Não Autorizado (Backdoor): O núcleo do programa (`packet_loop` e `shell`) aguarda por comandos remotos ocultos através de pacotes de rede e abre uma shell interativa (`/bin/sh`) conectada a um socket para conceder controle total ao invasor.

Técnicas de Ocultação (Rootkit/Stealth): Ele altera o nome do processo em execução (`set_proc_name`) para se disfarçar de daemons legítimos do sistema (como `udevd` , `dbus-daemon` , `auditd` , etc.) e manipula carimbos de tempo e arquivos de PID (`/var/run/haldrunner.pid`) para passar despercebido por administradores e ferramentas de monitoramento básicas. C + 1